Mikroişlemciler Vize — Konu Anlatımı

Doç. Dr. Bilal Babayiğit · BZ309 · ders notları, iskelet notlar ve çıkmış sorulardan derlendi.

KONU 1 · Hafta 1

Mikroişlemci Temelleri ve Tarihçe

> Sınavın en "sözel" kısmı burası. Her yıl 4–6 soru bu konudan geliyor: **ilk işlemci, ilk 32 bit, Power Wall, CISC/RISC, ARM, L1/L2/L3, mikroişlemci–mikrodenetleyici farkı**.

1.1 Mikroişlemci (MPU) – Mikrodenetleyici (MCU) – SoC

CPU: Komutları çalıştıran; aritmetik ve mantık işlemlerini yapan merkezi işlem birimi.
Mikroişlemci: CPU'nun tek bir entegre devre (çip) üzerinde gerçekleştirilmiş hâli. Bellek ve G/Ç birimleri **dışarıdadır**.

	Mikroişlemci (MPU)	Mikrodenetleyici (MCU)	SoC
İçerik	Sadece CPU; RAM, ROM, G/Ç yonga dışında	CPU + RAM + ROM + G/Ç **tek yongada**	CPU + GPU + NPU + bellek denetleyicisi + modem... tek silikonda
Amaç	Genel amaçlı, işletim sistemi çalıştırır, yüksek performans	Belirli bir göreve adanmış (gömülü sistem), düşük maliyet/güç	Telefon/tablet gibi cihazda komple sistem
Mimari	Genelde Von Neumann	Genelde Harvard	Karma
Örnek	PC işlemcisi (Intel Core, AMD Ryzen)	Arduino (ATmega), PIC, STM32, ESP32	Apple M serisi, Snapdragon

Sınavda nasıl soruldu?

- "Mikroişlemci nedir?" → **Bilgisayarın merkezi işlem birimidir; bellek ve G/Ç birimlerini içermez.** ("Kendine entegre bellek ve G/Ç bulunan" şıkkı mikrodenetleyicinin tanımıdır, tuzaktır.)
- "Hangisi MPU–MCU farklarından değildir?" → "Mikrodenetleyici çok yüksek hafıza, hız ve işlem gereken yerlerde tercih edilir" (yanlış; MCU minimal ve özel işler içindir).
- **Arduino** = temelde bir mikrodenetleyici kartı (basit gömülü projeler). **Raspberry Pi** = Linux çalıştırabilen tek kartlı bilgisayar (SoC). **PIC** = bir mikrodenetleyicidir. **PLC** = endüstriyel otomasyon; Raspberry Pi eğitim/prototip.

1.2 Bellek Mimarileri: Von Neumann – Harvard

Von Neumann

CPU ⇄ tek yol ⇄ Bellek (komut + veri)

Komut ve veri **aynı bellekte**, **tek ortak yol**. Basit ve ucuz. Komut ve veri aynı anda taşınamaz → **Von Neumann darboğazı**.

Harvard

CPU ⇄ yol 1 ⇄ Komut belleği ⇄ yol 2 ⇄ Veri belleği

Komut ve veri belleği **fiziksel olarak ayrı**, ayrı yollar → eş zamanlı erişim, darboğaz yok. Daha karmaşık/pahalı.

Günümüz = Değiştirilmiş (hibrit) Harvard: Dışarıdan tek RAM → Von Neumann gibi. Çekirdeğin içinde **L1 komut (L1-I)** ve **L1 veri (L1-D)** önbelleği ayrı → çekirdek içinde Harvard.

1.3 Tarihsel Süreç (zaman çizelgesi)

1971 · Intel 4004
İlk ticari (tek yongalı) mikroişlemci. **4 bit**, ~2300 transistör. Busicom **hesap makinesi** için yapıldı ama en önemli özelliği **programlanabilir** (genel amaçlı) olmasıydı.

1972 · Intel 8008
4004'ün **8 bit** versiyonu. 16 KB adresleme, 48 komut.

1974 · Intel 8080
8 bit, 64 KB adresleme, Altair 8800. Aynı dönemde **Zilog Z80** (Intel'den ayrılanlar kurdu), **Motorola 6800**, **MOS 6502** (çok ucuz → Apple II, Commodore 64, NES). **8085** (1977): Intel'in son 8 bit genel amaçlı işlemcisi.

1978 · Intel 8086
16 bit, 16 bit veri yolu, **20 bit adres yolu → 1 MB**. **x86 mimarisinin babası.**

1979 · 8088 / Motorola 68000
8088: içi 16 bit, dış veri yolu **8 bit** (eski 8 bit donanım/yazılımla uyum). **IBM PC 8088'i kullandı** → x86 standart oldu. 68000: 32 bit dâhili mimari, grafik arayüz çağı (Macintosh, Amiga, Atari ST, Sega Mega Drive).

1982 · 80286
24 bit adres → **16 MB**. Korumalı modu ilk sunan; **hem gerçek hem korumalı modda** çalışabilir. (Eski sınav anahtarı: "sanal bellek içeren ilk Intel = 80286".)

1985 · 80386
İlk 32 bit x86, 32 bit adres → **4 GB**, ~275.000 transistör. Tam **korumalı mod + sayfalama (paging)** → çok görevli işletim sistemleri (Windows, Linux).

1989 · 80486
>1 milyon transistör, **FPU (matematik işlemci) ve 8 KB önbellek yonga içinde**.

1993 · Pentium
AMD "AM386/AM486" üretince Intel rakamları tescil ettiremedi → 586 yerine **Pentium** (penta = 5). **Süperskalar**: bir saat vuruşunda birden fazla komut. 64 bit dış veri yolu.

1995 · Pentium Pro
36 bit adres → 64 GB.

2000 · AMD Athlon
1 GHz'i aşan ilk işlemci. 2003: AMD **x86'ya 64 biti getirdi** (AMD64 / x86-64).

~2005–2006 · Core Duo
Power Wall nedeniyle **çok çekirdeğe** geçiş (Core Duo, Core 2 Duo).

2021+ · Hibrit çekirdek
Intel P-Core/E-Core (Alder Lake, Meteor Lake), Apple M serisi, ARM big.LITTLE.

Sahneden çekilen devler – dersler

- **Zilog Z80**: 8 bitin kralıydı. IBM'in Intel'i seçmesi ve x86'nın yükselişi + 16/32 bite geçişte geride kalması → kayboldu.
- **Motorola 68000**: Macintosh/Amiga'nın kalbiydi. x86'nın PC dünyasındaki ezici hâkimiyeti; Apple da Intel'e geçti.
- **MOS 6502**: Çok ucuz ve verimli; ama Intel gibi devlerin Ar-Ge bütçesine yetişemedi (Commodore pazar adaptasyonunu sağlayamadı).
- **ARM**: **Acorn RISC Machine** (Acorn Computers). Düşük güç hedefi; çip üretmek yerine **mimari lisans satar** → başarılı oldu.
- Çıkarılan dersler: *Teknik üstünlük yeterli değildir · Ekosistem her şeydir · Adaptasyon ya da ölüm · Doğru zamanda doğru karar.*

Sınav tuzağı: "Hangisi mikroişlemci üreticisi/firması değildir?" → **Dell** (bilgisayar toplar), **Army**. Freescale, Zilog, Fairchild, National Instruments, Intel, AMD, Motorola şık olarak gelebilir.

1.4 Fiziksel Engeller: Moore, Power Wall, Dennard

- **Moore Yasası** (Gordon Moore, 1965): Yongadaki transistör sayısı yaklaşık **her 2 yılda ikiye katlanır**.
- **Frekans yarışı**: 2000'de AMD Athlon 1 GHz'i aştı. Frekans ↑ → güç tüketimi ↑↑ → ısı ↑↑.

Frekans ↑ → Güç ↑↑ → Isı ↑↑ → **Power Wall** 3–4 GHz tavan

Power Wall (Güç Duvarı): Frekans artırıldıkça işlemcinin soğutulamayacak kadar ısınması (erime noktası) fiziksel sınırı. Bu yüzden saat hızları **3–4 GHz** civarında kaldı. **Sınav sorusu**

Dennard ölçeklemesinin çöküşü (~2005): Eskiden transistör küçüldükçe voltaj/akım da orantılı düşer, **güç yoğunluğu sabit** kalırdı. Silikon sınırında kaçak akımlar başladı, voltaj daha fazla düşürülemedi.

Çözüm → Çok çekirdek: Tek çekirdeği hızlandırmak yerine aynı yongaya birden fazla çekirdek (Intel **Core Duo**). Yeni felsefe: *"Daha hızlı değil, daha fazla"* — performans artık paralel kaç iş yapıldığına bağlı.

1.5 CISC – RISC, Heterojen Çekirdek, RISC-V

	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Felsefe	"Donanım akıllı olsun": tek komutla karmaşık iş (bellekten oku + işle + yaz)	"Donanım basit ama çok hızlı olsun": basit komutların birleşimi
Temsilci	**Intel / AMD x86**, Motorola 68000	**ARM**, RISC-V, MIPS
Transistör / güç	Çok transistör, yüksek güç ve ısı	Az transistör, **düşük güç**, yüksek enerji verimliliği

- **ARM** işlemciler genellikle **düşük güç tüketimli cihazlar** için tasarlanır ve Intel'den **daha az enerji** tüketir.
- **Heterojen çekirdek** (big.LITTLE / P-Core + E-Core): Tüm çekirdekler aynı güçte değil. **P-Core**: oyun, render, ağır hesap. **E-Core**: arka plan işleri, boşta bekleme; pil tasarrufu, ısınmayı önler. Intel Meteor Lake'in öne çıkan yeniliği de budur.
- **RISC-V**: Telif ücreti olmayan, **açık kaynak** komut seti; **UC Berkeley** geliştirdi.
- VLIW (Very Long Instruction Word), EPIC (Itanium) de şıklarda görülür; Intel x86 = CISC.

1.6 İşlemci Performansını Belirleyen Faktörler

Performans **sadece GHz ile belirlenemez**.

1. **Çekirdek ve iş parçacığı (thread) sayısı** — aynı anda kaç bağımsız iş. **SMT / Hyper-Threading**: bir fiziksel çekirdeğin iki iş parçacığını aynı anda yürütmesi (aşçı fırını beklerken ikinci yemeğe başlar).
2. **Saat hızı** (Base / Turbo) — saniyedeki saat çevrimi. Tek başına anlamsızdır çünkü bir çevrimde ne kadar iş yapıldığı (**IPC**, Instructions Per Cycle) da önemlidir. Periyot = 1/f (2 GHz → 0,5 ns).
3. **Çekirdek dağılımı** (heterojen P/E).
4. **Önbellek** (L1/L2/L3).
5. **Komut seti mimarisi (ISA)** — x86, ARM...
6. **Üretim teknolojisi** (nm). Intel 2024: Intel 4 (7 nm sınıfı) → Intel 20A (2 nm sınıfı) geçişi.
7. **Paralellik**: Süperskalar (bir vuruşta çok komut) + Pipelining (boru hattı: getir → çöz → yürüt aşamaları üst üste).

Bir mikroişlemcinin ayırt edici özellikleri: saat hızı, veri yolu genişliği (bus width), transistör sayısı, adreslenebilir bellek. ("Bit per second transmit" ayırt edici özellik değildir.)

1.7 Önbellek (Cache) Hiyerarşisi

RAM, işlemciye göre çok yavaştır. Çekirdeğin belleği beklememesi için çok hızlı ara bellekler kullanılır.

Yazmaç

L1 — ceket cebi · KB · çekirdeğin içinde · I/D ayrı

L2 — sırt çantası · KB–MB · çekirdeğe özel

L3 — araba bagajı · onlarca MB · tüm çekirdekler paylaşır

RAM — GB · anakart üzerinde · yavaş

Yukarı çıktıkça **hız ↑, kapasite ↓**. L1 en hızlı ve en küçük, L3 en yavaş ve en büyük.

Çıkmış: "L1, L2, L3 veri erişim süresi?" → **L1 en hızlı, L3 en yavaş**. "En küçük kapasite?" → **L1**. 2023'te "Intel L1 boyutu?" diye belirsiz bir soru geldi; çekirdek başına tipik değer 32 KB komut + 32 KB veri = **64 KB** (yeni nesillerde 48 KB veri).

1.8 Araştırma/ödev konuları (sınavda sözel gelebilir)

- **SIMD vektör yazmaçları**: MMX (1996, 64 bit, MM0–7) → SSE (128 bit, XMM) → AVX/AVX2 (256 bit, YMM) → **AVX-512** (512 bit, ZMM0–31). Tek komutla çok veri; multimedya, 3D, bilimsel hesap ve **yapay zekâ (LLM) çıkarımı**. 64 bit bir işlemcinin 512 bit yazmacı olabilir çünkü "64 bit" genel amaçlı yazmaç/adres genişliğini anlatır; vektör yazmaçları ayrı birimlerdir.
- **Telefon/PC işlemci raporu**: mimari, çekirdek sayısı, saat, L1/L2/L3, üretim teknolojisi.

KONU 2 · Hafta 2

Sayı Sistemleri, ASCII, IEEE 754, Endianness

> Hesaplama sorularının kaynağı. **IEEE 754 her sınavda en az 1–2 soru**; bayrak (CF/OF) ve tümleme soruları da çok sık.

2.1 İkili (binary) – Onaltılı (hex)

- **8 bit = 1 bayt. 4 bit = 1 hex basamağı** (nibble) → binary'yi 4'erli grupla.
- Hex yazımı: `0x413F` veya `413Fh`. Assembly'de harfle başlayan hex'in önüne 0 konur: `0F2h`, `0FFFFh`.

0100	0001	0011	1111
4	1	3	F

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	11

Örnekler: 149 = 1001 0101b = 95h · 1101 0110b = D6h = 214 · B7h = 1011 0111b = 183 · **100 = 64h** (MOV AL,100 → AL = 64h).

Kesirli kısım: 2 ile çarp, tam kısmı yaz. 0,375 → 0,75(0) → 1,5(1) → 1,0(1) ⇒ **0,011b**. 0,6875 = 0,1011b. 0,25 = 0,01b. 0,625 = 0,101b.

2.2 İşaretsiz ve İşaretli Tam Sayılar

	Aralık (n bit)	8 bit	16 bit
İşaretsiz	$0 \dots 2^n-1$	0 … 255	0 … 65535
İşaretli (ikiye tümleyen)	$-2^{n-1} \dots 2^{n-1}-1$	−128 … +127	−32768 … +32767

İşaret-büyüklük yöntemi (MSB işaret, kalanı büyüklük) kusurludur: **iki sıfır** (+0 = 0000 0000, −0 = 1000 0000) ve toplama bozulur (+5 + (−5) = 1000 1010 = −10).

> **İkiye tümleyen (2's complement):** Çıkarmayı ek devre olmadan toplayıcıyla yapar.
> 1) Pozitif ikili yaz → 2) Tüm bitleri ters çevir (bire tümleyen) → 3) +1 ekle.
> +5 = 0000 0101 → 1111 1010 → **1111 1011 = −5** (FBh).
> MSB (en yüksek değerlikli bit) **işaret bitidir**: 1 ise negatif.

Hex'te hızlı yol: negatif = (FF..F − sayı) + 1. Örn. 16 bit −1 = FFFFh, −2 = FFFEh.

2.3 Elde (Carry) ve Taşma (Overflow)

	Carry Flag (CF)	Overflow Flag (OF)
Hangi aritmetik?	**İşaretsiz**	**İşaretli**
Anlamı	MSB'den dışarı elde çıktı (toplama) / ödünç alındı (çıkarma)	Sonuç işaretli aralığa sığmadı, işaret biti bozuldu
Donanım	CF = C_n (son toplayıcının elde çıkışı)	OF = $C_{n-1} \oplus C_n$ (XOR kapısı)

OF kısa yolu: İki **pozitif** toplanıp sonuç **negatif** çıkarsa ya da iki **negatif** toplanıp **pozitif** çıkarsa OF = 1. Farklı işaretli iki sayının toplamında asla taşma olmaz.

Örnek (+70)+(+80): 0100 0110 + 0101 0000 = 1001 0110 = −106 (işaretli) → **OF=1**, CF=0 (işaretsiz 150 ≤ 255 sığar).

Örnek (hoca sorusu): A=5Ah (+90), B=4Ch (+76). A+B = **A6h** = 1010 0110 → işaretli −90. CF=0 (8 bitte dışarı elde yok), **OF=1** (iki pozitif toplandı, sonuç negatif; 166 > 127).

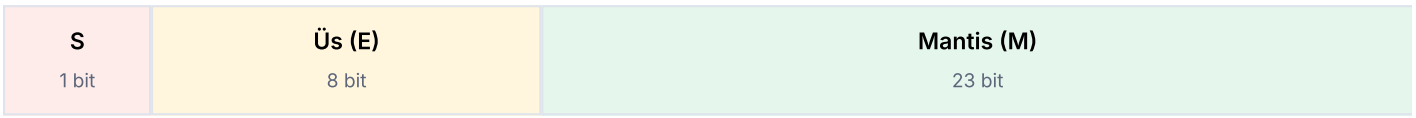
2023 vizesi:
• AX=7FFFh (32767) + BX=1 → 8000h. İşaretsiz elde yok → **CF = 0 (sıfır olur)**; işaretli taşma var → OF=1. (Birçok öğrenci "CF set edilir" işaretledi — yanlış.)
• AX=8000h + BX=8000h → 1 0000h → AX=0000h, CF=1, ZF=1, OF=1, düşük bayt 00h'da 1 sayısı çift (0) → **PF = 1 (set edilir)**.

2.4 Karakter Kodlama

- **ASCII**: **7 bit**, 128 karakter. PC'lerde kullanılan klasik standart. Sadece temel Latin.
- 'A' = 65 = **41h**, 'a' = 97 = **61h**, '0' = 48 = **30h**, boşluk = 20h, **CR (satır başı) = 0Dh**, **LF (alt satır) = 0Ah**. CR+LF = yeni satır (\n).
- Büyük–küçük harf farkı **32 = 20h** = **5. bit**. `XOR AL, 20h` → harfin büyüklüğünü tersine çevirir; `AND AL, 0DFh` → büyük harf; `OR AL, 20h` → küçük harf.
- Rakam karakteri → sayı: `SUB AL, 30h` ; sayı → karakter: `ADD AL, 30h` .
- **Genişletilmiş ASCII**: 8 bit, 256 karakter.
- **Unicode**: Tüm diller, simgeler, emojiler. **UTF-8**: **değişken uzunluklu** (1–4 bayt), ASCII ile geriye uyumlu (ASCII karakterleri 1 bayt). (Eski notlarda "Unicode 16 bit" der — UTF-16'yı kastediyor.)

2.5 Kayan Noktalı Sayılar – IEEE 754

İkili bilimsel gösterimde normalize sayı her zaman **1.xxxx × 2^n** şeklindedir; baştaki 1 saklanmaz (**gizli bit**), yalnızca virgülden sonrası (mantis) saklanır.



	Tek duyarlıklı (float, 32 bit)	Çift duyarlıklı (double, 64 bit)
İşaret / Üs / Mantis	1 / 8 / 23	1 / 11 / 52
Bias	**127**	**1023**
Değer	$(-1)^S \times 1.M \times 2^{E-127}$	$(-1)^S \times 1.M \times 2^{E-1023}$

Ondalık → IEEE 754 (5 adım)

Örnek: −12,25

1. İşaret: negatif → **S = 1**
2. İkiliye çevir: 12 = 1100, 0,25 = 0,01 → **1100,01**
3. Normalize: $1{,}10001 \times 2^3$ → gerçek üs 3
4. Saklanan üs: 3 + 127 = 130 = **1000 0010**
5. Mantis: 10001 + 18 sıfır → 1 | 1000 0010 | 1000 1000...0

Bitleri 4'erli grupla: 1100 0001 0100 0100 0000... = **0xC1440000**

Diğer örnekler: **9,75** = 1001,11 = $1{,}00111 \times 2^3$ → **0x411C0000** · **−12,6875** = 1100,1011 → **0xC14B0000** · **10,625** = 1010,101 → 0 10000010 0101010... = **0x412A0000** · **11,0** = 0x41300000.

IEEE 754 → Ondalık

Örnek: 0xBE200000

1. Binary: 1011 1110 0010 0000 ...
2. S = 1 (negatif); E = 0111 1100 = 124 → gerçek üs = 124 − 127 = **−3**; M = 0100...
3. Sayı = $-1{,}01b \times 2^{-3} = -1{,}25 \times 0{,}125 =$ **−0,15625**

Diğerleri: 0x411E0000 = **+9,875** · 0x41300000 = **11,0** · 0xC00C0000 = $-1{,}00011b \times 2^1$ = **−2,1875** (2024 çıkmışı).

Hızlı kontrol: 0x41... ile başlayan pozitif sayılar 8–16 arası (üs 3), 0x40... 2–8 arası (üs 1–2), 0x3F... 0,5–2 arası, 0xC... negatif. Sitedeki IEEE 754 aracı ile adım adım dene.

2.6 Endianness (Bayt Sıralaması)

Bellek **bayt adreslemelidir**. Çok baytlı bir veri belleğe hangi sırayla yazılır?

	Little Endian (küçük sonlu)	Big Endian (büyük sonlu)
Kural	En düşük değerli bayt (LSB) **en küçük adrese**	En yüksek değerli bayt (MSB) en küçük adrese
Kullanan	**Intel / AMD x86**, modern ARM varsayılanı	Motorola 68k, SPARC, **ağ (TCP/IP) – "Network Byte Order"**

0x1A2B3C4D → 2000..2003	4D 3C 2B 1A	1A 2B 3C 4D

Analoji: Gün/Ay/Yıl (little) – Yıl/Ay/Gün (big). ARM aslında **bi-endian** (iki modu da destekler) ama genelde little kullanılır.

KONU 3 · Hafta 3

Bellek Yönetimi: Gerçek Mod, Korumalı Mod, Sayfalama

> "Bellek yönetimi neden gerekli, nasıl hesaplanır?" — her sınavda 3–5 soru. **Fiziksel adres formülü** ve **mod–bellek sınırı** tablosunu ezberle.

3.1 Adres hattı ↔ bellek kapasitesi

n adres hattı → 2^n farklı bayt adreslenebilir.

Adres yolu	Kapasite	İşlemci / mod
16 bit	64 KB	Tek 16 bit yazmacın görebildiği alan (bir segment)
20 bit	**1 MB**	8086/8088 – **gerçek mod**
24 bit	16 MB	80286
32 bit	**4 GB**	80386+ – **korumalı mod** (32 bit Windows'ta 2×8 GB RAM taksan da 4 GB görürsün)
36 bit	64 GB	Pentium Pro (PAE)
40 bit	1 TB	Kitaptaki (Brey) 64 bit flat mod bilgisi – eski sınavlar
48 bit	256 TB	Günümüz x86-64 (4 seviyeli sayfalama); sunucular 57 bit

Adres yolu **tek yönlüdür** (işlemciden belleğe/G/Ç'ye). Veri yolu **çift yönlüdür**. 8086'da veri yolu **16 bit**.

3.2 Gerçek Mod (Real Mode) – Segment:Offset

Problem: 8086'nın yazmaçları 16 bit (en fazla 64 KB görür) ama adres yolu 20 bit (1 MB).
Çözüm: Bellek en fazla **64 KB**'lık **segmentlere** bölünür. Adres = **Segment** (bloğun başlangıcı) + **Offset** (blok içindeki uzaklık, 16 bit).

```
Fiziksel Adres = Segment × 16 + Offset = (Segment << 4) + Offset
```

Hex'te ×16 = **sağa bir 0 eklemek** (4 bit sola kaydırma).

Örnekler

- 1234h:0010h → 12340h + 0010h = **12350h**
- CS = A000h, IP = 1234h → A0000h + 1234h = **A1234h** (sonraki komutun adresi)
- 1000:1F00 → 10000h + 1F00h = **11F00h**

- SS = 2400h, SP = 0100h, `PUSH AX` → SP = 0100h − 2 = 00FEh → fiziksel 24000h + 00FEh = **240FEh**

Bellek örtüşmesi (aliasing): Farklı segment:offset çiftleri aynı fiziksel adresi gösterebilir: 0000:0100 = 0010:0000 = 00100h. Segmentler 16 baytlık sınırlardan (paragraf) başlar.

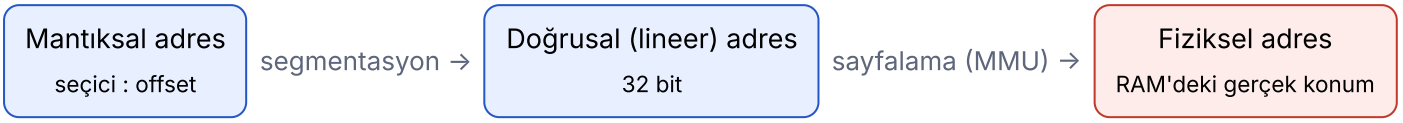
Segment – offset yazmaç çiftleri

İş	Segment	Offset	Açıklama
Komut getirme	**CS** (Code)	**IP**	Sonraki komutun adresi; IP otomatik artar
Yığın	**SS** (Stack)	**SP**, BP	PUSH/POP, dönüş adresi, yerel değişkenler
Veri	**DS** (Data)	BX, SI, DI	Değişkenler, diziler
Ek veri	**ES** (Extra)	DI	String/kopyalama işlemlerinde hedef

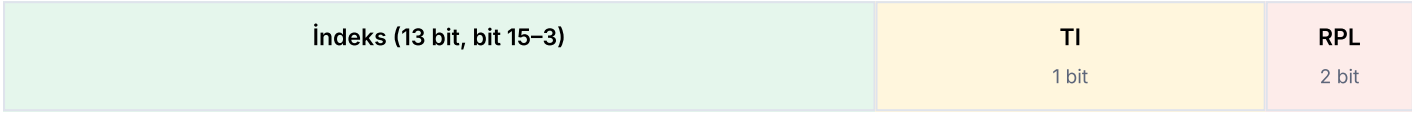
4 segment yazmacı: **Code, Data, Stack, Extra** (sonradan FS, GS eklendi). Gerçek modda **sayfalama yoktur**, sadece segmentasyon; adres doğrudan fiziksel adrestir. 8086/8088/80186 sadece gerçek modda; **80286 ve üstü hem gerçek hem korumalı modda** çalışabilir.

3.3 Korumalı Mod (Protected Mode, 80286/80386+)

Neden? Gerçek modda bellek koruması yok: bir program diğerinin (ya da işletim sisteminin) belleğine yazıp sistemi çökertebilir. Korumalı mod her programın sadece kendine izin verilen alana erişmesini donanımla sağlar → çok görevli, kararlı işletim sistemleri.

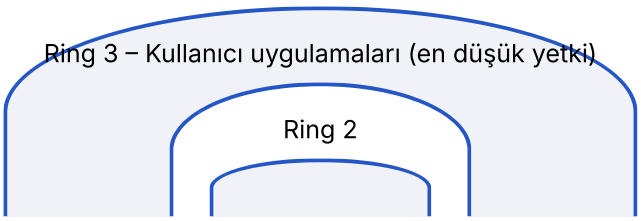


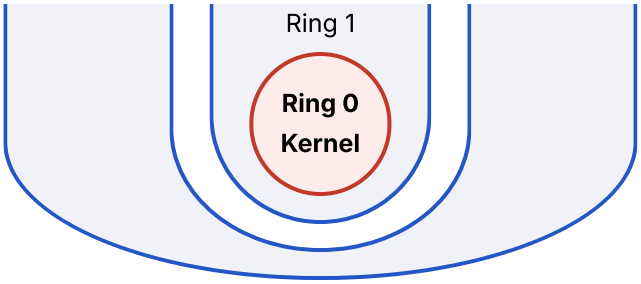
- Segment yazmaçları artık taban adres değil, **seçici (selector)** tutar: tablodaki bir girdinin indeksi. Segment yazmacı → tablo → tanımlayıcı → taban adres.
- **Tanımlayıcı (descriptor)**: 8 baytlık girdi. İçinde segmentin **taban adresi (32 bit)**, **limiti (boyutu)**, **erişim hakları** (kod: çalıştırılabilir; veri: okunur/yazılır) ve türü.
- **GDT** (Global Descriptor Table): sistem geneli / işletim sistemi. **LDT** (Local): her göreve özel.



- 13 bit indeks → 2^{13} = **8192** tanımlayıcı. TI=0 → GDT, TI=1 → LDT. RPL: istenen ayrıcalık (2 bit → 4 seviye).
- Korumalı modda (32 bit) bir program 16 bit seçici + 32 bit offset kullanır, sonuç **32 bit doğrusal adres**. Maksimum **4 GB**.

Koruma halkaları (Ring)





Ring 1–2 sürücüler içindi, modern sistemlerde kullanılmıyor. Yetki aşımı → **GPF (General Protection Fault)**.

Korumalı mod yazmaçları: **GDTR, LDTR, IDTR, TR**; kontrol yazmaçları **CR0–CR4**.

3.4 Sayfalama (Paging) ve Sanal Bellek

- Programın gördüğü (sanal) adres uzayı eşit bloklara = **sayfa**; fiziksel RAM aynı boyutta **çerçevelere (frame)** bölünür. Sayfa boyutu genelde **4 KB**.
- **Sayfa tablosu**: hangi sanal sayfanın hangi fiziksel çerçevede olduğunu tutar. Sayfalar RAM'de **ardışık olmak zorunda değil** → belleğin verimli kullanımı.
- **Sanal belleğin amacı**: fiziksel RAM'den **daha büyük** bellek alanı sağlamak (disk takas alanı ile).

Neden 4 KB? — 10-10-12 kuralı (32 bit)

Sayfa dizini	Sayfa tablosu	Offset
10 bit	10 bit	12 bit

Offset 12 bit → 2^{12} = 4096 = **4 KB**. $2^{10} \times 2^{10} \times 4$ KB = **4 GB** = 2^{32}.

Sayfa hatası (Page Fault, kesme 14)

Sayfa tablosu girdisinde **P (present)** biti vardır. P = 0 olan sayfaya erişilirse MMU **sayfa hatası** üretir. İşletim sistemi: kesmeyi yakalar → sayfayı diskteki takas alanından okur → boş bir çerçeveye yükler → tabloyu günceller (P=1) → durdurulan komutu yeniden çalıştırır.

Donanım birimleri

- **MMU** (Memory Management Unit): her erişimde sayfa tablolarını kullanarak sanal/doğrusal adresi fiziksele çevirir.
- **TLB** (Translation Look-aside Buffer): son çevrilen adreslerin önbelleği. TLB hit → tek çevrimde çeviri; miss → CR3'ten tabloları tara.
- **CR0**: **PE** biti korumalı modu, **PG** biti sayfalamayı açar. **CR3**: aktif görevin sayfa dizininin fiziksel taban adresi.

3.5 64 Bit ve Flat (Düz) Bellek Modeli

- Segmentasyon büyük ölçüde terk edildi; program belleği **tek parça** görür, yönetim neredeyse tamamen **sayfalama** ile.
- CS, DS, ES, SS tabanları donanımda **0** kabul edilir (CS hâlâ kod segmenti ve ayrıcalık bilgisi için kullanılır). **FS ve GS** istisna: **Thread Local Storage (TLS)** gibi işletim sistemi verileri için.
- 64 bit modda **gerçek mod yoktur**. Segment yazmaçları bellek konumunu adreslemek için kullanılmaz.
- Eski sınav anahtarı (Brey): "64 bit flat modda adres 40 bittir, 1 TB adresler." Günümüzde sanal adres 48 bit (256 TB).

Mod	Adresleme	Adres	Bellek
Gerçek mod (8086)	Segment:Offset	20 bit	1 MB
Korumalı mod (386)	Segmentasyon + sayfalama	32 bit	4 GB
64 bit mod	Flat + sayfalama	48 bit (57 sunucu)	256 TB

3.6 A20 Gate (ödev)

8086'da adres 1 MB'ı aşınca başa sarar (FFFFFh → 00000h, *wrap-around*); bazı DOS programları buna güveniyordu. 80286'nın 24 bit adres yolunda taşma olmayınca bu programlar çöktü. Çözüm: 21. adres hattı (**A20**) klavye denetleyicisindeki boş bir pinle açılıp kapatılabilen bir kapıya bağlandı; eski yazılımlar için A20 = 0 yapılarak taşma taklit edildi.

KONU 4 · Hafta 4

Yazmaçlar (Register) ve Bayraklar

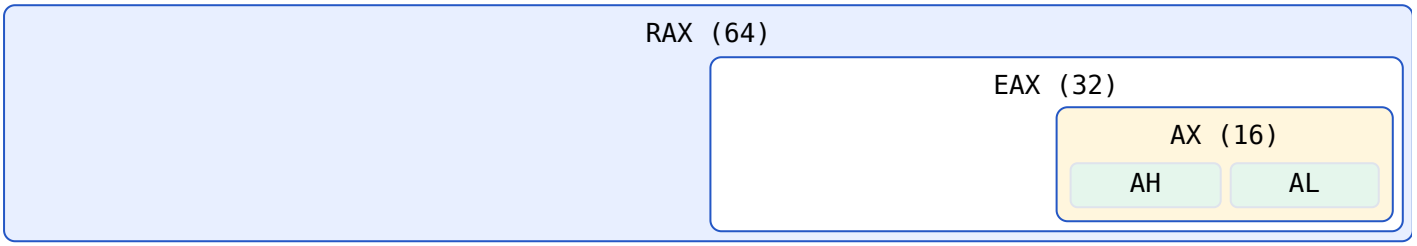
> Neredeyse her sınavda: "Hangisi 32 bitlik/8 bitlik yazmaçtır?", "Hangisi genel amaçlı yazmaç değildir?", "RAX/EBX/RSP ne için kullanılır?", "ZF ne zaman set edilir?"

4.1 Neden yazmaç?

RAM büyük ama yavaş ("kereste deposu"), yazmaç küçük ama işlemcinin kendi frekansında çalışan en hızlı bellek ("çalışma masası"). Verimli program: **RAM'den yazmaca yükle → işlemi yazmaçta yap → sonucu RAM'e yaz.** Hız sırası: **Yazmaç > Önbellek > RAM > Disk**.

4.2 Genel amaçlı yazmaçlar

64 bit	32 bit	16 bit	8 bit (yüksek / düşük)	Geleneksel rol
RAX	EAX	AX	AH / AL	**Accumulator** – aritmetik/mantık, fonksiyon dönüş değeri
RBX	EBX	BX	BH / BL	**Base** – bellek adreslemede taban
RCX	ECX	CX	CH / CL	**Counter** – döngü sayacı (LOOP), kaydırma sayısı (CL)
RDX	EDX	DX	DH / DL	**Data** – çarpma/bölmede yüksek kısım ve **kalan**, G/Ç port adresi
RSI	ESI	SI	SIL	Source Index – kaynak adres
RDI	EDI	DI	DIL	Destination Index – hedef adres
RBP	EBP	BP	BPL	Base Pointer – yığındaki yerel değişkenlere taban
RSP	ESP	SP	SPL	Stack Pointer – **yığının tepesi**
R8–R15	R8D	R8W	R8B	Sadece 64 bitte var (yeni 8 yazmaç)



- Ön ekler: **E** = Extended (32), **R** = Register (64). R8–R15 için sonekler: **D** = Dword (32), **W** = Word (16), **B** = Byte (8). Örn. **R8D 32 bit**, **R8W 16 bit**, **R15B 8 bit**.
- **R7 diye bir yazmaç yoktur** (yeni yazmaçlar R8'den başlar) → "Hangisi genel amaçlı yazmaç değildir?" cevabı.
- Assembly büyük/küçük harf duyarsız: `rax, RAX, Rax` aynı.
- 64 bit modda 32 bitlik bir yazmaca yazmak (`mov eax, …`) RAX'in üst 32 bitini **sıfırlar**; 8/16 bitlik yazma ise üst kısmı değiştirmez.

Byte	Word	Dword (double word)	Qword (quad word)
8 bit	16 bit / 2 bayt	32 bit / 4 bayt	64 bit / 8 bayt

4.3 Özel amaçlı yazmaçlar

- **RIP** (Instruction Pointer): bir sonraki çalıştırılacak komutun adresi. "CPU şimdi ne yapacak?" sorusunun cevabı.
- **RSP**: yığının en üstteki adresi. PUSH ile **azalır**, POP ile artar.
- **RFLAGS** (16 bit: FLAGS, 32 bit: EFLAGS): son işlemin sonucuna dair bayrak bitleri → karar verme (if, koşullu atlama).
- Segment yazmaçları: CS, DS, SS, ES, FS, GS. Kontrol: CR0–CR4.

4.4 Bayraklar (Flags)

Bayrak	Ne zaman 1?	Örnek
ZF Zero	Sonuç **sıfır** ise	`SUB EAX,EAX` → ZF=1 (eşitlik kontrolü: CMP)
SF Sign	Sonuç negatif (MSB = 1)	`MOV EAX,-5` sonra `TEST EAX,EAX` → SF=1
CF Carry	**İşaretsiz** taşma: MSB'den elde / çıkarmada ödünç	AL=255, `ADD AL,1` → AL=0, CF=1, ZF=1
OF Overflow	**İşaretli** taşma: işaret biti bozuldu	AL=127, `ADD AL,1` → AL=80h, OF=1, SF=1
PF Parity	Sonucun **düşük baytındaki 1 sayısı çift** ise	Sonuç 00h → PF=1; 01h → PF=0
AF	Bit 3'ten bit 4'e elde (BCD)	
DF, IF, TF	Yön, kesme izni, adım adım (trap)	8086'da toplam 9 bayrak

Bayrakları etkilemeyen komutlar: MOV, LEA, PUSH, POP, XCHG, JMP/Jcc, LOOP, CALL/RET, NOT. **INC/DEC** CF'yi etkilemez (diğer bayrakları etkiler). ADD, SUB, CMP, AND, OR, XOR, NEG ZF/SF/PF'yi günceller (AND/OR/XOR CF=OF=0 yapar).

Çıkmış: "AX'e 5 değerini ata, ZF ne olur?" — beklenen cevap **ZF sıfır olur** (sonuç sıfır değil). Teknik not: MOV bayrakları hiç değiştirmez; hoca "sonuç sıfır değil" mantığını soruyor.

"ZF'yi etkileyen işlem hangisi? (add, cmp, and, inc, or)" — aslında **hepsi** ZF'yi etkiler; soru hatalıydı. MOV/LEA/PUSH ZF'yi etkilemez.

İşaretsiz kullanım: bellek adresleri, dizi indisleri, döngü sayaçları, RGB (0–255). **İşaretli**: finans, fizik simülasyonu, oyunlar, hata kodları.

4.5 Boyut uyumu kuralları (MOV hedef, kaynak)

Komut		Neden
`MOV EAX, EBX`	✓	32 ← 32
`MOV RAX, 10`	✓	Sabit hedefin boyutuna uyar
`MOV EAX, RBX` / `MOV AL, BX`	✗	operand size mismatch
`MOV AL, 1FFh`	✗	1FFh 8 bite sığmaz
`MOV [2000], 5`	✗	Boyut belirsiz → `MOV DWORD PTR [2000], 5`
`MOV [2000], [3000]`	✗	Bellekten belleğe doğrudan MOV yok; yazmaç üzerinden
`MOV EAX, [RBX]`	✓	RBX'teki adresten 4 bayt oku

KONU 5 · Hafta 4

Adresleme Modları, Veri Tanımlama ve Yığın

> Hoca: "Adresleme modları **hem vizede hem finalde** çıkar." Her yıl 3–5 soru: modu tanıma, dizi elemanının adresini hesaplama, LEA vs MOV.

Adresleme modu: komutun işleyeceği verinin **nerede** olduğunu işlemciye söyleme yöntemi. Temel komut: `MOV hedef, kaynak` (kaynağı hedefe kopyalar, kaynak değişmez).

Örnek bellek (int = 4 bayt)		
2000	**5**	*global_x*
3000	**1**	*s.a*
3004	**7**	*s.b*
4000	**10**	*arr[0]*
4004	**20**	*arr[1]*
4008	**30**	*arr[2]*
4012	**40**	*arr[3]*

5.1 Modlar

#	Mod	Örnek	Veri nerede?	C karşılığı
1	**Register** (yazmaç)	`MOV RAX, RBX`	Başka bir yazmaçta. Belleğe gidilmez → **en hızlı**	`y = x;`
2	**Immediate** (anlık/ivedi)	`MOV EAX, 10`	Komutun içinde sabit sayı	`int x = 10;`
3	**Direct** (doğrudan)	`MOV EAX, [2000h]`	Adres komutta açıkça yazılı	global değişken
4	**Register Indirect** (dolaylı)	`MOV EAX, [RBX]`	Adres bir yazmaçta (BX, SI, DI, BP...)	`y = *p;`
5	**Register Relative** (taban + öteleme)	`MOV EAX, [RBX+4]`	Yazmaçtaki adres + sabit öteleme (displacement)	`s.b`, struct alanı, yığın değişkeni `[RBP+8]`

6	**Base-Indexed** (taban + indeks)	`MOV AX, [BX+SI]`	Taban yazmacı + indeks yazmacı	2 boyutlu erişim
7	**Relative Base-Indexed** (göreli taban indeksli)	`MOV AX, [BP+SI+04h]`	Taban + indeks + sabit	dizi içinde struct
8	**Scaled Index** (ölçekli indeks)	`MOV EAX, [RSI+RCX*4]`	taban + indeks × ölçek + öteleme	`x = arr[i];`

`[ base + index × scale + displacement ]  ·  scale ∈ {1, 2, 4, 8}`

Ölçek = dizideki bir elemanın bayt boyutu: char 1, short 2, int 4, long/pointer/double 8. Scaled index **dizi elemanlarına, aralarındaki sabit artışlarla erişmek** için kullanılır; en güçlü ve esnek mod.

C'den Assembly'ye: `int arr[] = {10,20,30,40}; int i = 2; int x = arr[i];` — dizi adresi RSI'da, i RCX'te:

```
MOV EAX, [RSI + RCX*4]   ; 4000 + 2*4 = 4008 → EAX = 30
```

Çıkmışlardan:

- `MOV AX,[BP+SI+04H]` → **Göreli Taban İndeksli (Relative Base-Indexed)**.
- Dolaylı adresleme örneği → `MOV AX,[BX]` . (`[1234H]` doğrudan, `1234H` anlık, `BX` yazmaç, `[SI+BX]` taban-indeksli.)
- Register indirect'te "kaydedici içindeki değer, **verinin adresini** gösterir".
- Register adreslemenin avantajı → **bellek kullanımını azaltır, program hızını artırır**.
- Scaled index'te ölçek → **1, 2, 4 veya 8**.

5.2 Veri tanımlama ve adres hesabı

Direktif	Anlamı	Eleman boyutu
`db`	define byte	1 bayt
`dw`	define word	2 bayt
`dd`	define double word	4 bayt
`dq`	define quad word	8 bayt

`?` = başlangıç değeri belirsiz (yer ayrılır). Değişkenler bellekte **tanımlandıkları sırayla art arda** yerleşir.

Her yıl gelen soru kalıbı

```
notlar   dw 69H, 87H, 96H, 45H, 75H
enyuksek dw ?
```

- **notlar 1000H'de ise enyuksek nerede?** 5 eleman × 2 bayt = 10 = 0Ah → **100AH**. (2024)
- Aynı soru `dd` ile: 5 × 4 = 20 = 14h → **1014H**. (2023)
- **3. elemanın adresi**: notlar + 2×boyut → dw: notlar+4, dd: notlar+8.
- **3. elemanın değeri** (96H) RAX'e: `LEA RBX, notlar` → `MOV AX, [RBX+4]` (dw). 64 bite genişletmek için `MOVZX RAX, WORD PTR [RBX+4]`. Sınavda "en doğru" şık `Lea rbx, notlar / Mov rax, word ptr [rbx+4]` idi.
- `Sayilar dw 34,45,56,67,75,89` → 6 × 2 = **12 bayt** yer.

LEA ve MOV farkı

- `MOV RAX, [RBX+8]` → o adresteki **veriyi** okur.
- `LEA RAX, [RBX+8]` → sadece **adresi hesaplar** (RBX+8), belleğe gitmez. `LEA RCX, mybyte` → RCX = mybyte'ın **adresi**. (`MOV RAX, OFFSET notlar` da adres verir.)
- LEA hızlı çarpma için de kullanılır: `LEA EAX,[EAX+EAX*2]` = EAX×3, `LEA EAX,[EAX+EAX*4]` = ×5. Tek komutta yalnızca **bir** ölçekli indeks olabilir (`[eax*2+eax*3]` geçersiz). LEA, IMUL'dan hızlıdır ve bayrakları etkilemez.

5.3 Yığın (Stack): PUSH / POP

- Yığın **aşağı doğru büyür**: `PUSH` önce SP/RSP'yi **azaltır**, sonra veriyi yazar. `POP` veriyi okur, sonra RSP'yi **artırır**. LIFO (son giren ilk çıkar).
- 16 bit: PUSH AX → SP −= 2. 64 bit: PUSH RAX → RSP −= **8**.
- 64 bit modda: `PUSH imm` (ör. `PUSH 0BEBAFECAh`) **8 bayt** yer kaplar (32 bit sabit 64'e işaret genişletilir); `PUSH AX` / `PUSH WORD PTR …` 2 bayt; `PUSH EAX` ve `PUSH AL` **geçersizdir** (sadece 16 ve 64 bit push).
- Değer belleğe **little endian** yazılır.

Örnek (64 bit): RSP = ...FEE0h iken

```
MOV RAX, 123456789ABCDEF0H
PUSH RAX      ; RSP = ...FED8
MOV RBX, 0FEDCBA9876543210H
PUSH RBX      ; RSP = ...FED0
MOV RCX, 0F0F0F0F0F0F0F0FH
PUSH RCX      ; RSP = ...FEC8   (3 × 8 = 24 = 18h azaldı)
```

Örnek (final tipi): `MyByte dw 1234H,5678H` , RSP = 0938H.
`PUSH 0BEBAH` → RSP = 0930H, [0930]: BA BE 00 00 00 00 00 00
`PUSH WORD PTR [MyByte]` → RSP = **092EH**, [092E]: 34 12

KONU 6 · Hafta 4–5 + çıkmışlar

Sık Çıkan Komutlar ve Kesmeler

> Çıkmışlarda kısa kod parçası verip "AX/BX ne olur?", "Ekranda ne görünür?" diye soruluyor. Bu sayfadaki her komutun tek satırlık anlamını bilmek yeterli.

6.1 Veri taşıma ve aritmetik

Komut	İş	Not
`MOV a, b`	a ← b	Bayrakları etkilemez
`ADD a, b` / `SUB a, b`	a ← a ± b	CF, OF, ZF, SF, PF
`INC a` / `DEC a`	±1	**CF'yi etkilemez** (bu yüzden ADD varken INC var)
`CMP a, b`	a − b yapar, sonucu yazmaz	Sadece bayraklar → sonrasında Jcc
`NEG a`	ikiye tümleyen (−a)	
`IMUL r`	RDX:RAX ← RAX × r	İşaretli çarpma; sonuç büyükse üst kısım RDX
`IMUL r1, r2` / `IMUL r1, r2, imm`	r1 ← r1×r2 / r1 ← r2×imm	
`IDIV r`	bölünen / r	**Bölüm AX/EAX/RAX'te, kalan DX/EDX/RDX'te**

Bölen boyutu	Bölünen	Bölüm	Kalan
byte	AX	AL	AH
word (ör. `IDIV CX`)	DX:AX	AX	**DX**
dword	EDX:EAX	EAX	EDX
qword	RDX:RAX	RAX	RDX

6.2 Mantık, kaydırma, döndürme

`AND / OR / XOR / NOT`	Bit bit işlemler. `XOR AX,0FFFFh` = tüm bitleri ters çevirir (1234h → **EDCBh**). `XOR AX,AX` = sıfırla.

`SHL a, n`	Sola kaydır, sağdan 0 gelir → **×2^n**
`SHR a, n`	Sağa kaydır, soldan 0 gelir → **÷2^n** (işaretsiz). 00FFh SHR 2 = 003Fh (4'e bölündü)
`ROL a, n`	Sola döndür: soldan çıkan bitler sağdan girer. 0F0Fh ROL 4 = **F0F0h**
`ROR a, n`	Sağa döndür. 10100110b ROR 3 = **11010100b**

Yazmacı sıfırlama yolları: `MOV AX,0` · `SUB AX,AX` · `XOR AX,AX` · `AND AX,0` · yeterince `SHL` . **ADD ile sıfırlama yapılamaz.**

6.3 Akış kontrolü

- `JMP etiket` → koşulsuz olarak **bir bellek adresine atlama**.
- Koşullu: `JE/JZ` (ZF=1), `JNE/JNZ` , `JG/JL/JGE/JLE` (işaretli), `JA/JB` (işaretsiz), `JC` (CF=1).
- `LOOP etiket` = `DEC CX/RCX` + `JNZ etiket` : CX sıfır olana kadar etikete döner.
- `CALL` : dönüş adresini yığına atar ve alt programa gider; `RET` : yığından adresi alıp döner.

6.4 Giriş/Çıkış ve DOS/BIOS kesmeleri (8086)

- `IN AL, 80h` → 80h portundan oku. `OUT 80h, AL` → AL'yi **80h portuna yaz**. Port 255'ten büyükse DX'te: `OUT DX, AL` .

Kesme	AH	İş
INT 21h	01h	Klavyeden karakter oku (ekrana yansıtır), AL'ye
INT 21h	02h	**DL**'deki karakteri ekrana yaz
INT 21h	07h/08h	Karakter oku, ekrana yansıtmaz
INT 21h	09h	DS:DX'teki **$ ile biten dizeyi (string) ekrana yazar**
INT 21h	4Ch	Programı sonlandır
INT 10h	0Eh	AL'deki karakteri ekrana yaz (teletype)
INT 16h	00h	**Bir tuş basılmasını bekler**, ASCII kodunu AL'ye (tarama kodunu AH'ye) döndürür

2025 sorusu:

```
        mov cx, 3
print_loop:
        mov ah, 0Eh
        mov al, '*'
        int 10h        ; *
        mov al, 0Dh
        int 10h        ; CR: satır başı
        mov al, 0Ah
        int 10h        ; LF: alt satır
        loop print_loop
```

Döngü 3 kez → her satıra bir yıldız, alt alta 3 satır. (Sadece LF olsaydı yıldızlar **merdiven** gibi kayarak inerdi.)

KONU 7 · Hafta 5

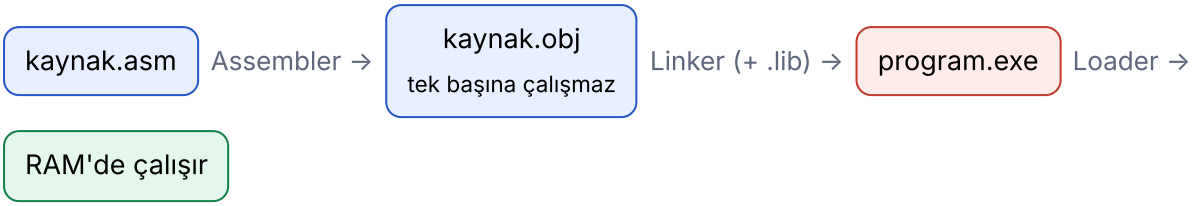
Assembly Programlama Ortamı (Toolchain)

> Notlarda "**Bir soru gelir**" ve "Visual Studio'da C++ ve Assembly → **soru gelir**" yazıyor. Çıkmışlarda her yıl assembler adı, oluşan dosya ve araçların görevi soruluyor.

7.1 Fikirden koda, koddan makineye

Analoji – yemek tarifi: **Assembly kodu (.asm)** şefin yazdığı okunabilir tarif; **işlemci** sadece bitmiş yemeği yiyebilen müşteri; **toolchain** tarifi pişirip sunan mutfak ekibi.

Toolchain (araç zinciri): Kaynak kodu (.asm, .cpp) işletim sisteminin çalıştırabileceği dosyaya (.exe) dönüştüren programlar dizisi (assembler, compiler, linker, loader, debugger). Bir programlama ortamı (IDE) değildir.



1. **Assembler (çevirici)**: Assembly kodunu (.asm) **makine diline** (0/1) çevirir. Çıktı: **object file (.obj)**; henüz tek başına çalıştırılamaz. Ayrıca liste dosyası (.lst) üretebilir.
2. **Linker (bağlayıcı)**: Bir veya daha fazla .obj ile kütüphaneleri (.lib) birleştirip "bağlantıları" kurar → **.exe**. Örn. kodunuz printf çağırıyorsa linker printf'in makine kodunu bulup birleştirir. (.map dosyası da linker'ın çıktısıdır.)
3. **Loader (yükleyici)**: .exe'yi diskten **RAM'e yükler** ve çalıştırır (işletim sisteminin parçası).
4. **Debugger (hata ayıklayıcı)**: Programı **satır satır** çalıştırıp durdurur; o anki **yazmaç ve bellek** değerlerini gösterir. Hatayı anlamanın en güçlü aracı.

Çıkmış: "MASM/TASM ornek.asm (enter) yaparsanız hangi dosya oluşur?" → **ornek.obj** (sonra LINK ornek.obj → ornek.exe).
"Assembler, compiler ve debugger" → **farklı işlevlere sahiptir ve program geliştirmenin farklı aşamalarında kullanılır.**
"___ CPU'nun yürütebilmesi için assembly programlarını makine diline çevirir" → **Assembler**.

7.2 Derleyici (Compiler) – Yorumlayıcı (Interpreter)

	Derleyici	Yorumlayıcı
Çalışma	Kaynağın **tamamını tek seferde** makine koduna çevirir; .exe kendi başına çalışır	Kaynağı **satır satır** okur, her çalıştırmada anında çevirip çalıştırır
Hız	Hızlı	Daha yavaş
Örnek	C, C++, Go, Rust, (Assembly → assembler)	Python, JavaScript, PHP, Ruby

Java (ödev): hibrittir. `javac` kaynağı **bytecode**'a derler; bytecode **JVM** tarafından yorumlanır/JIT ile derlenir → "bir kez yaz, her yerde çalıştır".

7.3 Assembler programları

MASM	Microsoft Macro Assembler (64 bit: `ml64.exe`); linker'ı **LINK**. Bu derste kullanılan.
TASM	Turbo Assembler (Borland; Pascal/Delphi'nin firması)
NASM	Netwide Assembler – açık kaynak, çok platformlu
FASM	Flat Assembler – çok küçük, hızlı, kendi kendini derler
GAS	GNU Assembler (as) – Linux/GCC, **AT&T** sözdizimi
YASM, JWASM, WASM (Watcom)	Diğerleri
CASM	**Böyle bir assembler yok** — "hangisi assembler değildir?" sorusunun cevabı (her yıl soruluyor!)

7.4 Intel vs AT&T sözdizimi (araştırma)

	Intel (bu derste)	AT&T (GCC/Linux)
Operand sırası	`mov rax, rbx` → **hedef, kaynak**	`movq %rbx, %rax` → **kaynak, hedef**
Yazmaç	rax	**%**rax
Sabit	5	**$**5
Boyut	QWORD PTR	komut soneki: b, w, l, q
Bellek	[rbx+rcx*4+8]	8(%rbx,%rcx,4)

7.5 Assembly'nin yeri

- Assembly komutları genellikle **bellek ve yazmaç manipülasyonunu** temsil eder.
- Avantaj: hızlı, küçük programlar, donanıma en yakın kontrol; C/C++ ile yapılamayan bazı şeyler yapılabilir. Dezavantaj: programlaması zor, **işlemciye özeldir** — farklı işlemciler **aynı assembly dilini kullanmaz** (x86, ARM, MIPS ayrı).
- Kullanım alanı: **donanım sürücüleri**, işletim sistemi çekirdeği, gömülü sistemler, performans kritik küçük fonksiyonlar.
- "MIPS, x86, ARM" terimleri → **assembly dilleri ve mimariler** bağlamında.

7.6 Visual Studio'da C++ + Assembly (sınıf uygulaması)

Modern ortamda Assembly tek başına değil, C/C++ ile birlikte kullanılır: C++ karmaşık mantığı yönetir, Assembly performansın en kritik olduğu küçük fonksiyonlarda "cerrahi müdahale" yapar.

main.cpp

```
#include <iostream>
// Assembly fonksiyonunu C++'a tanıt.
// extern "C": isim karıştırmayı (name mangling) kapatır
extern "C" int ToplamaFonksiyonu(int a, int b);

int main() {
    int sonuc = ToplamaFonksiyonu(10, 20);
    std::cout << "Sonuc: " << sonuc; // 30
    return 0;
}
```

islemler.asm (MASM x64)

```
.code
PUBLIC ToplamaFonksiyonu
ToplamaFonksiyonu PROC
    ; 1. parametre (a) -> RCX (ECX)
    ; 2. parametre (b) -> RDX (EDX)
    mov eax, ecx      ; dönüş değeri EAX/RAX
    add eax, edx
    ret               ; C++'a geri dön
ToplamaFonksiyonu ENDP
END
```

Komut satırından derleme:

```
ml64.exe /c islemler.asm        → islemler.obj
cl.exe   /c main.cpp            → main.obj
link main.obj islemler.obj /SUBSYSTEM:CONSOLE  → main.exe
```

Kod çalışmaz, derlenir ve bağlanır → Debugger en iyi dostunuzdur → Modern assembly bir takım oyunudur.

KONU 8 · Hafta 6

Çağırma Kuralı (Calling Convention)

> Son işlenen konu. C++ ve Assembly'nin birbiriyle "konuşabilmesi" için uyulması gereken kurallar: parametreler nereye konur, sonuç nereden okunur, hangi yazmaçlar korunmalı?

8.1 Microsoft x64 (Windows) çağırma kuralı

Parametre	64 bit (Qword)	32 bit (Dword)	16 bit	8 bit
1.	**RCX**	ECX	CX	CL
2.	**RDX**	EDX	DX	DL
3.	**R8**	R8D	R8W	R8B
4.	**R9**	R9D	R9W	R9B
5. ve sonrası	**Yığın (stack)**			

- **Dönüş değeri**: tam sayı/işaretçi **RAX** (int için EAX); ondalık (float/double) XMM0. Ondalık parametreler XMM0–XMM3.
- **Shadow space (gölge alan)**: Çağıran, CALL'dan önce yığında 4 parametre için **32 bayt** yer ayırır.
- CALL anında yığın **16 bayta hizalı** olmalıdır. `CALL` dönüş adresini (8 bayt) yığına iter; `RET` onu alıp geri döner.

8.2 Yazmaç sorumluluğu

Grup	Yazmaçlar	Kural
Volatile (caller-saved, uçucu)	RAX, RCX, RDX, R8, R9, R10, R11	Fonksiyon bunları serbestçe bozabilir; çağıran değerini korumak istiyorsa kendisi saklar.
Non-volatile (callee-saved, kalıcı)	**RBX, RBP, RDI, RSI, RSP, R12–R15**	Fonksiyon bunları kullanacaksa önce yığına **PUSH**'layıp çıkarken **POP** ile geri yüklemelidir; doğrudan değiştirilmemelidir.

Linux (System V) farkı: ilk 6 parametre RDI, RSI, RDX, RCX, R8, R9; dönüş yine RAX. Sınav Windows/MASM üzerinden.

8.3 Örnekler

İki sayının küçüğü

```
enkucuk PROC          ; int enkucuk(int a, int b)
    mov  eax, ecx     ; eax = a
    cmp  edx, eax
    jg   Bitti        ; b > a ise a kalsın
    mov  eax, edx     ; değilse eax = b
Bitti:
    ret               ; sonuç EAX'te
enkucuk ENDP
```

Üç sayının küçüğü (ödev)

```
enkucuk3 PROC         ; (ecx, edx, r8d)
    mov  eax, ecx
    cmp  edx, eax
    jg   Atla1
    mov  eax, edx
Atla1:
    cmp  r8d, eax
    jg   Atla2
    mov  eax, r8d
Atla2:
    ret
enkucuk3 ENDP
```

RBX'i kullanmak isteyen bir fonksiyon:

```
Fonk PROC
    push rbx          ; callee-saved → koru
    mov  rbx, rcx
    ...               ; işlemler
    mov  rax, rbx     ; sonuç
    pop  rbx          ; geri yükle
    ret
Fonk ENDP
```